

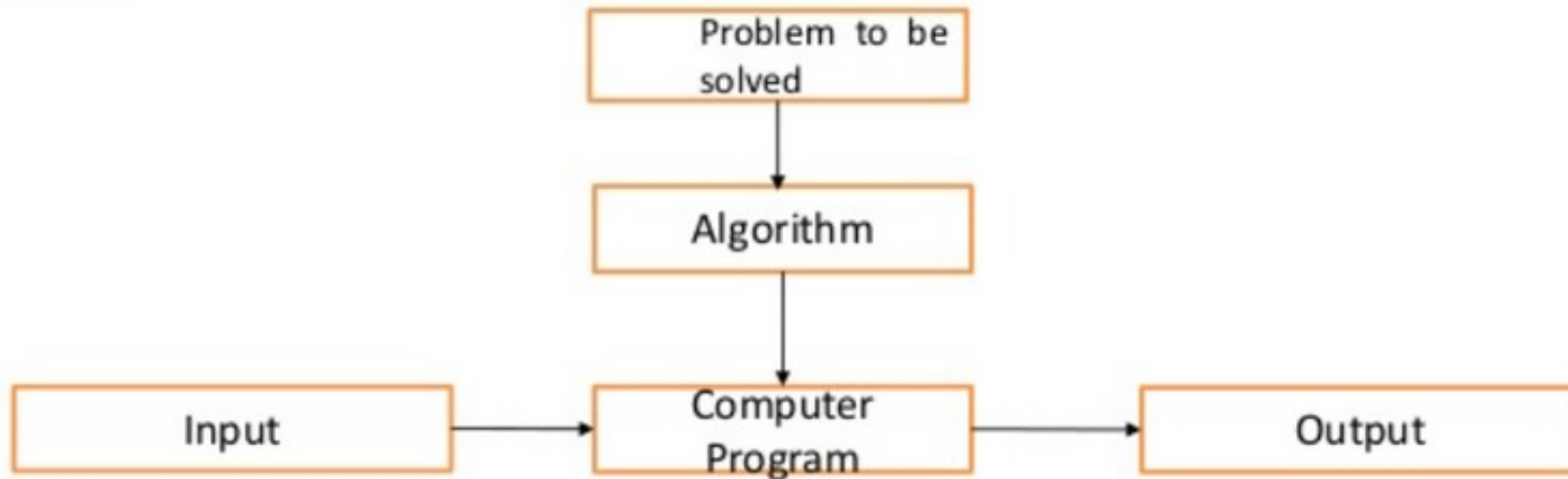
Design and Analysis of Algorithms

-Introduction

Introduction to Algorithm

NOTION OF AN ALGORITHM

An ***algorithm*** is a sequence of unambiguous instructions for solving a problem, i.e., for obtaining a required output for any legitimate input in a finite amount of time.



Goals of Algorithm Study

- To develop framework for instructing computer to perform tasks
- To introduce notion of algorithm as means of specifying how to solve a problem
- To introduce and appreciate approaches for defining and solving very complex tasks in terms of simpler tasks;

Algorithm - Definition

- An algorithm for solving a problem
 - “a *finite* sequence of *unambiguous*, *executable* steps or instructions, which, if followed would ultimately *terminate* and give the solution of the problem”.
- Note the keywords:
 - Finite set of steps;
 - Unambiguous;
 - Executable;
 - Terminates;

Algorithm/Pseudocode – find the max in an array

Algorithm

1. Set the temporary maximum equal to the first integer in the sequence.
2. Compare the next integer in the sequence to the temporary maximum, and set the larger one to be temporary maximum.
3. Repeat the previous step if there are more integers in the sequence.
4. Stop when there are no integers left in the sequence. The temporary maximum at this point is the maximum in the sequence.

Algorithm 1. Finding the Maximum Element in a Finite Sequence

//Input : n integers $a_1, a_2, \dots, a_n$

// Output : max (the maximum of $a_1, a_2, \dots, a_n$)

max := a_1 ;

for i : =2 to n

 if max < a_i then max := a_i

return max

Instead of using a particular computer language, we use a form of pseudocode.

Algorithm - Example

- **Problem: What is the largest integer?**

INPUT: All the integers { ... -2, -1, 0, 1, 2, ... }

OUTPUT: The largest integer

Algorithm:

- Arrange all the integers in a list in decreasing order;
 - MAX = first number in the list;
 - Print out MAX;
- WHY is the above NOT an Algorithm?
 - (Hint: How many integers are there?)

Example - ArrayMax

Algorithm arrayMax(A, n):

Input: An array A storing $n \geq 1$ integers.

Output: The maximum element in A .

$currentMax \leftarrow A[0]$

for $i \leftarrow 1$ **to** $n - 1$ **do**

if $currentMax < A[i]$ **then**

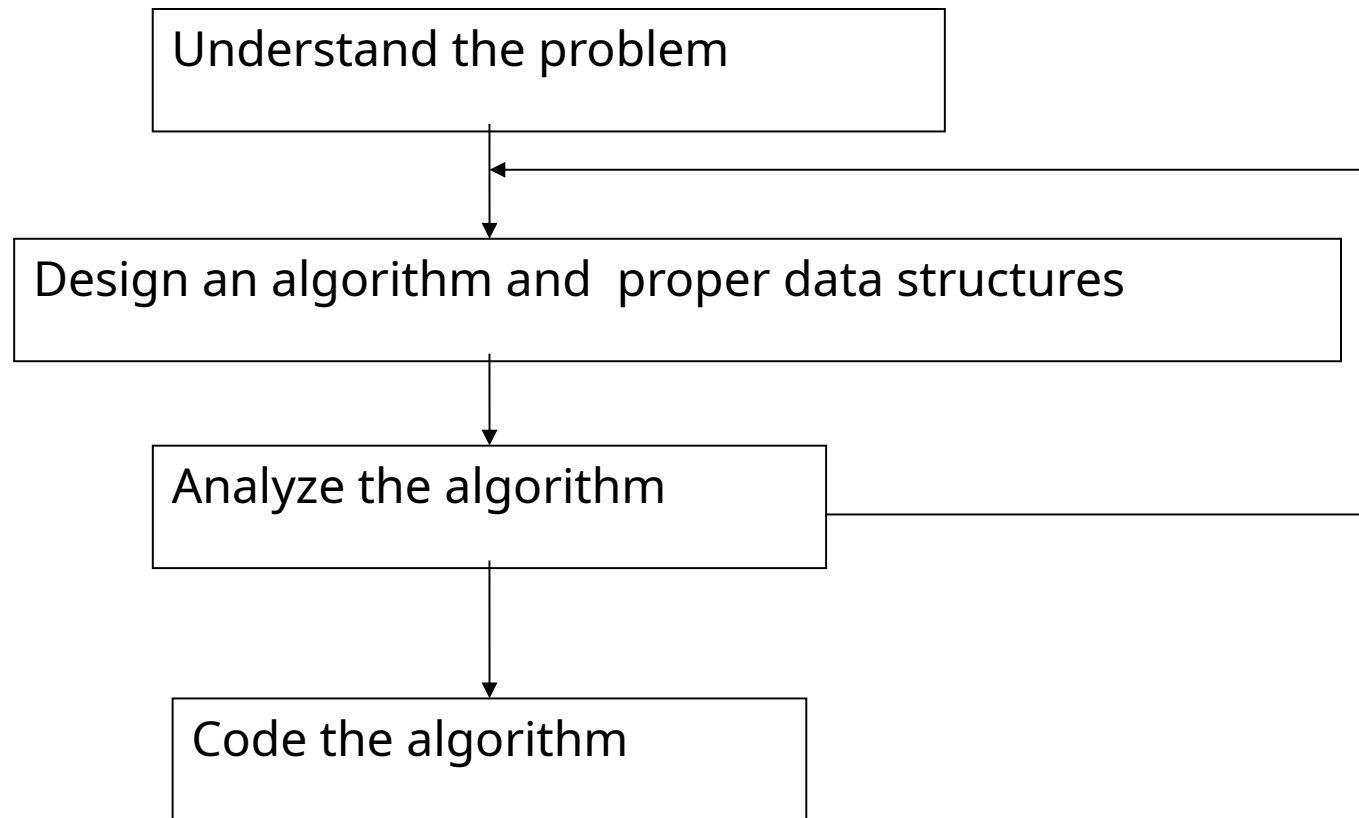
$currentMax \leftarrow A[i]$

return $currentMax$

Characteristics of an Algorithm

- ❑ **Input:** Zero / more quantities are externally supplied.
- ❑ **Output:** At least one quantity is produced.
- ❑ **Definiteness:** Each instruction is clear and unambiguous.
- ❑ **Finiteness:** If the instructions of an algorithm is traced then for all cases the algorithm must terminates after a finite number of steps.
- ❑ **Efficiency:** Every instruction must be very basic and runs in short time.

Fundamentals of Algorithmic Problem Solving



Important Problem Types

- Sorting
- Searching
- String processing (e.g. string matching)
- Graph problems (e.g. graph coloring problem)
- Combinatorial problems (e.g. maximizes a cost)
- Geometric problems (e.g. convex hull problem)
- Numerical problems (e.g. solving equations)

Some Algorithm Types

- Algorithm types include:
 - Simple recursive algorithms
 - Brute force algorithms
 - Divide and conquer algorithms
 - Greedy algorithms
 - Dynamic programming algorithms
 - Backtracking algorithms
 - Branch and bound algorithms
 - Randomized algorithms

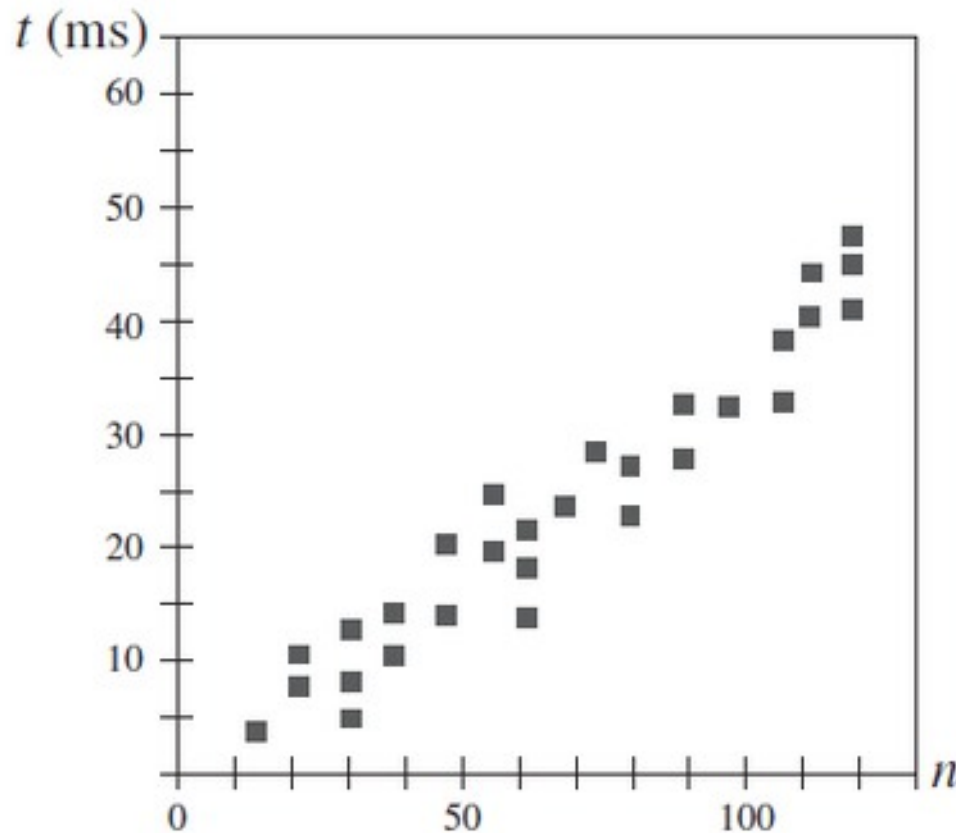
Types of Algorithm Analysis

- **Time complexity**: the time required to solve a problem of a specified size
 - The time complexity is expressed in terms of the number of operations used by the algorithm.
- **Space complexity**: the computer memory required to solve a problem of a specified size.
- **Analysis Types**
 - **Worst case analysis**: the largest number of operations needed to solve the given problem using this algorithm.
 - **Average case analysis**: the average number of operations used to solve the problem over all inputs.
 - **Best case analysis**: the minimum number of operations used to solve the problem over all inputs.

Fundamentals of the Analysis of the Algorithm

- Implementation and Empirical Analysis
- Challenge in empirical analysis:
 - Develop a correct and complete implementation.
 - Determine the nature of the input data and other factors influencing on the experiment. Typically, There are three choices: actual data, random data, or perverse data.
 - Compare implementations independent to programmers, machines, compilers, or other related systems.
 - Consider performance characteristics of algorithms, especially for those whose running time is big.
- Can we analyze algorithms that haven't run yet?

Empirical Algorithm Analysis



- Results of an experimental study on the running time of an algorithm.
- A dot with coordinates (n, t) indicates that on an input of size n , the running time of the algorithm is t milliseconds (ms).

Random Access Machine Model (RAM)

- Experimental analysis is valuable, but it has its limitations
- To analyze a particular algorithm without performing experiments on its running time, we can use RAM model
- A set of high-level primitive operations that are largely independent from the programming language used can be identified and analyzed
- Primitive operations include the following:
 - Assigning a value to a variable
 - Calling a method
 - Performing an arithmetic operation (for example, adding two numbers)
 - Comparing two numbers
 - Indexing into an array
 - Following an object reference
 - Returning from a method.

Example - ArrayMax

Algorithm arrayMax(A, n):

Input: An array A storing $n \geq 1$ integers.

Output: The maximum element in A .

$currentMax \leftarrow A[0]$ 1+1

for $i \leftarrow 1$ **to** $n - 1$ **do**

if $currentMax < A[i]$ **then**

$currentMax \leftarrow A[i]$

return $currentMax$

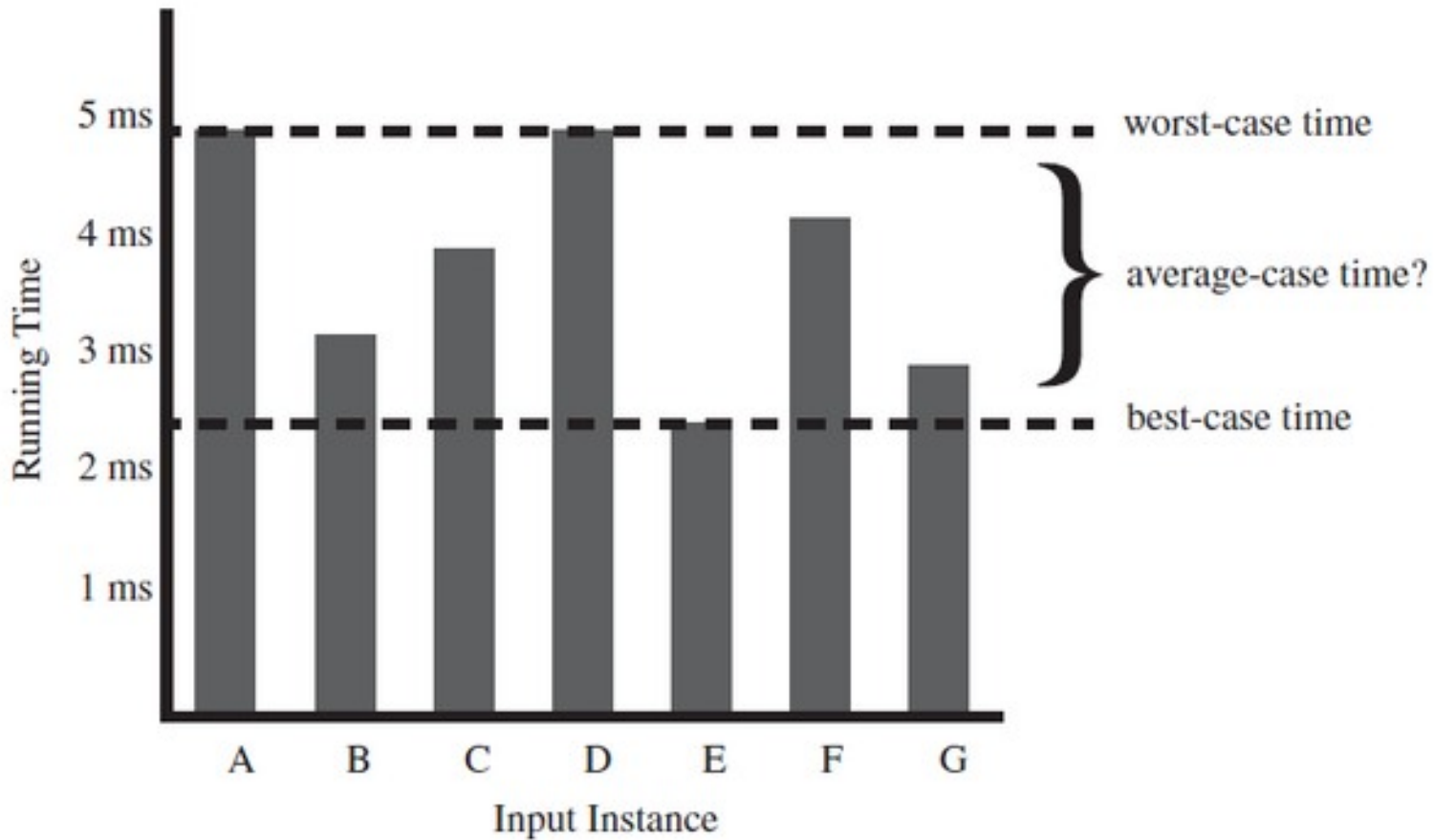
1 (init)+ [1(cond)+ 1(if-check)+1(array)
+1(array)+1(assign)+2 (incr+assign)]
(n-1) + 1(false cond) + 1(return)

This is called
recurrence equation

$T(n) = 7(n-1) + 5 = 7n - 2 \Rightarrow$ worst
case

$T(n) = 5n \Rightarrow$ best case

Best case and Worst case



Analyzing Recursive algorithms

Algorithm recursiveMax(A, n):

Input: An array A storing $n \geq 1$ integers.

Output: The maximum element in A .

if $n = 1$ **then**

1

return $A[0]$

1 + 1

return $\max\{\text{recursiveMax}(A, n - 1), A[n - 1]\}$

1+1+1+1+1+1

$$T(n) = \begin{cases} 3 & n = 1 \\ T(n-1) + 7 & \text{elsewhere} \end{cases}$$

**This is called
recurrence relation**

Try Yourself!!

- Write a Pseudocode to find the sum of first N natural numbers. Using the RAM model, find the recurrence equation
- Write a Pseudocode/Algorithm to find the factorial of a given number.
- For the above examples, try the recursive procedure too..
- Try to find the above examples for varying 'n's values and plot it using python plot function

Example – factorial of a number

Iterative

version

Fact(n)

Begin

if $n == 0$ or 1 then

return 1

else

f=1

for i in 2 to n do

f=f*I

end for

end if

End

Recursive version

Fact(n)

Begin

if $n == 0$ or 1 then

Return 1;

else

Return $n * \text{Call Fact}(n-1)$;

endif

End

Few examples

```
1  Algorithm Sum( $a, n$ )
2  {
3       $s := 0.0$ ;
4      for  $i := 1$  to  $n$  do
5           $s := s + a[i]$ ;
6      return  $s$ ;
7  }
```

```
1  Algorithm RSum( $a, n$ )
2  {
3      if ( $n \leq 0$ ) then return 0.0;
4      else return RSum( $a, n - 1$ ) +  $a[n]$ ;
5  }
```

Few examples

```
1  Algorithm Add(a, b, c, m, n)
2  {
3      for i := 1 to m do
4          {
5              count := count + 2;
6              for j := 1 to n do
7                  count := count + 2;
8              }
9          count := count + 1;
10 }
```

Generic Notations - Examples

```
1  Algorithm TowersOfHanoi( $n, x, y, z$ )
2  // Move the top  $n$  disks from tower  $x$  to tower  $y$ .
3  {
4      if ( $n \geq 1$ ) then
5          {
6              TowersOfHanoi( $n - 1, x, z, y$ );
7              write ("move top disk from tower",  $x$ ,
8                  "to top of tower",  $y$ );
9              TowersOfHanoi( $n - 1, z, y, x$ );
10         }
11 }
```

Generic Notations - Examples

```
procedure  $F(n)$   
  //returns the nth Fibonacci number//  
  integer  $n$   
  if  $n \leq 1$  then return( $n$ )  
    else return( $F(n - 1) + F(n - 2)$ )  
  endif  
end  $F$ 
```


Generic Notations - Examples

```
1: procedure  $F2(n)$ ;  
2: begin  
3:   if  $n > 0$  then  
4:      $m := \text{random}(0, n - 1)$ ;  
5:     for  $i := 1$  to  $n$  do  
6:        $F2(m)$ ;  
7:        $F2(m)$ ;  
8:     end for
```

```
9:     for  $i := 0$  to  $n$  do  
10:       $\text{print}(i)$ ;  
11:    end for  
12:  else  
13:     $\text{print}(n)$ ;  
14:  end if  
15: end
```

Algorithm Efficiency

- **Linear Loops**

for (i = 0; i < 1000; i++)

application code

- $T(n) = n$

for (i = 0; i < 1000; i += 2)

application code

- $T(n) = n / 2$

- If you were to plot either of these loop examples, you would get a straight line. For that reason they are known as linear loops.

Algorithm Efficiency

- Logarithmic Loops

Multiply Loops

**for (i=1; i<1000; i*=2)
application code**

Divide Loops

**for (i=1000; i>0; i/=2)
application code**

$$T(n) = \log n$$

Multiply		Divide	
Iteration	Value of i	Iteration	Value of i
1	1	1	1000
2	2	2	500
3	4	3	250
4	8	4	125
5	16	5	62
6	32	6	31
7	64	7	15
8	128	8	7
9	256	9	3
10	512	10	1
(exit)	1024	(exit)	0

Algorithm Efficiency

- Linear Logarithmic

```
for (i = 0; i < 10; i++)  
    for (j = 0; j < 10; j *= 2)  
        application code
```

$$T(n) = 10\log 10$$

$$T(n) = n\log n$$

Algorithm Efficiency

- Quadratic

```
for (i = 0; i < 10; i++)  
    for (j = 0; j < 10; j++)  
        application code
```

$$T(n) = n^2$$

- Dependent Quadratic

```
for (i = 0; i < 10; i++)  
    for (j = 0; j < i; j++)  
        application code
```

$$1 + 2 + 3 + \dots + 9 + 10 = 55$$

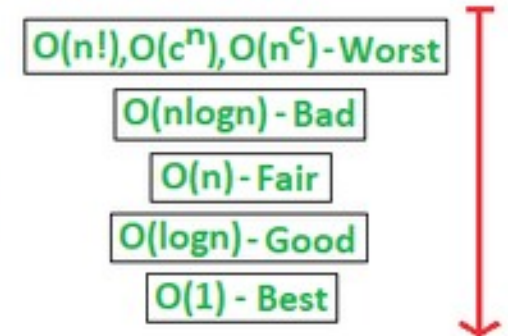
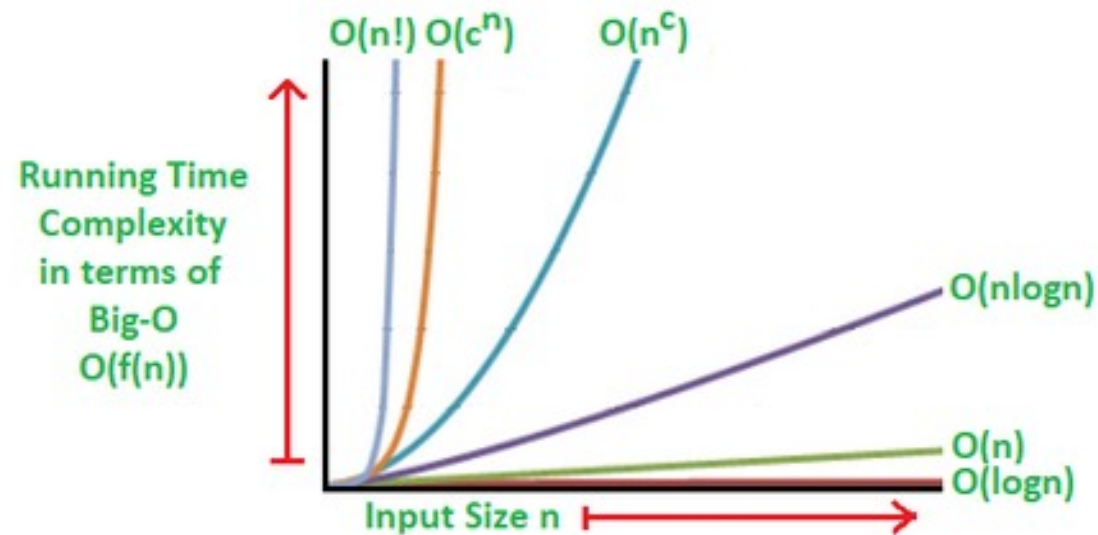
$$f(n) = n\left(\frac{n+1}{2}\right)$$

$$T(n) = n^2$$

Algorithm Efficiency – Overall comparison

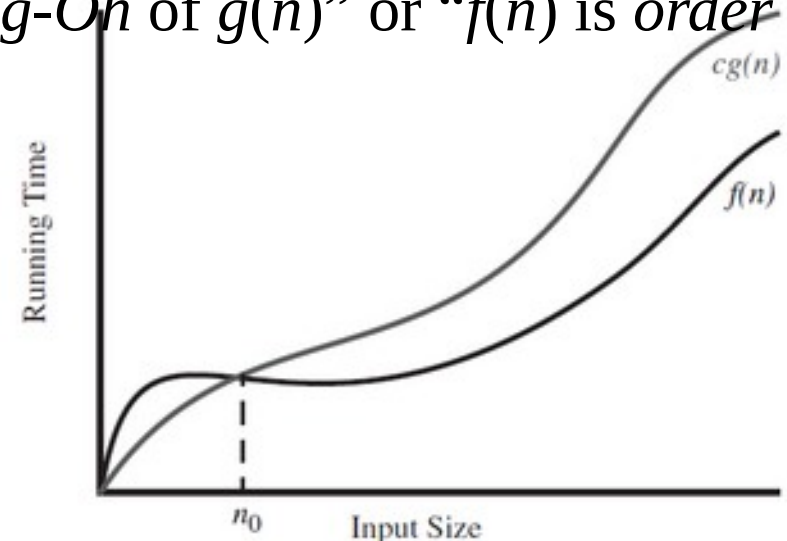
Efficiency	Big-O	Iterations	Estimated Time
Logarithmic	$O(\log n)$	14	microseconds
Linear	$O(n)$	10,000	seconds
Linear logarithmic	$O(n(\log n))$	140,000	seconds
Quadratic	$O(n^2)$	$10,000^2$	minutes
Polynomial	$O(n^k)$	$10,000^k$	hours
Exponential	$O(c^n)$	$2^{10,000}$	intractable
Factorial	$O(n!)$	$10,000!$	intractable

Assuming $n=10000$



Asymptotic Notations - The “Big-Oh” Notation

- Let $f(n)$ and $g(n)$ be functions mapping nonnegative integers to real numbers. We say that $f(n)$ is $O(g(n))$ if there is a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \leq cg(n)$ for every integer $n \geq n_0$.
- This definition is often pronounced as “ $f(n)$ is *big-Oh* of $g(n)$ ” or “ $f(n)$ is *order* $g(n)$.”
- Example: $7n - 2$ is $O(n)$.



Examples

Example 1: $7n - 2$ is $O(n)$

- We need a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $7n - 2 \leq cn$ for every integer $n \geq n_0$. It is easy to see that a possible choice is $c = 7$ and $n_0 = 1$, but there are other possibilities as well.

Example 2: $20n^3 + 10n \log n + 5$ is $O(n^3)$

Example 3: $3 \log n + \log \log n$ is $O(\log n)$

- $3 \log n + \log \log n \leq 4 \log n$, for $n \geq 2$. Note that $\log \log n$ is not even defined for $n = 1$, but $\log \log n < \log n$, for $n \geq 2$. That is why we use $n \geq 2$.

Example 4: 2^{100} is $O(1)$

Example 5: $5n \log n + 2n$ is $O(n \log n)$

Examples

Big – O - theorem

- Let $d(n)$, $e(n)$, $f(n)$, and $g(n)$ be functions mapping nonnegative integers to nonnegative reals.
 1. If $d(n)$ is $O(f(n))$, then $ad(n)$ is $O(f(n))$, for any constant $a > 0$.
 2. If $d(n)$ is $O(f(n))$ and $e(n)$ is $O(g(n))$, then $d(n)+e(n)$ is $O(f(n)+g(n))$.
 3. If $d(n)$ is $O(f(n))$ and $e(n)$ is $O(g(n))$, then $d(n)e(n)$ is $O(f(n)g(n))$.
 4. If $d(n)$ is $O(f(n))$ and $f(n)$ is $O(g(n))$, then $d(n)$ is $O(g(n))$.
 5. If $f(n)$ is a polynomial of degree d (that is, $f(n) = a_0 + a_1n + \dots + a_d n^d$), then $f(n)$ is $O(n^d)$.
 6. n^x is $O(a^n)$ for any fixed $x > 0$ and $a > 1$.
 7. $\log n^x$ is $O(\log n)$ for any fixed $x > 0$.
 8. $\log^x n$ is $O(n^y)$ for any fixed constants $x > 0$ and $y > 0$.

Example

- Prove by theorem : $2n^3 + 4n^2 \log n$ is $O(n^3)$.
 - $\log n$ is $O(n)$ (Rule 8).
 - $4n^2 \log n$ is $O(4n^3)$ (Rule 3).
 - $2n^3 + 4n^2 \log n$ is $O(2n^3 + 4n^3)$ (Rule 2).
 - $2n^3 + 4n^3$ is $O(n^3)$ (Rule 5 or Rule 1).
 - $2n^3 + 4n^2 \log n$ is $O(n^3)$ (Rule 4).

logarithmic	linear	quadratic	polynomial	exponential
$O(\log n)$	$O(n)$	$O(n^2)$	$O(n^k)$ ($k \geq 1$)	$O(a^n)$ ($a > 1$)

Big-Omega and Big-Theta

- Let $f(n)$ and $g(n)$ be functions mapping integers to real numbers. We say that $f(n)$ is $\Omega(g(n))$ (pronounced “ $f(n)$ is big-Omega of $g(n)$ ”) if $g(n)$ is $O(f(n))$; that is, there is a real constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $f(n) \geq cg(n)$, for $n \geq n_0$. This definition allows us to say asymptotically that one function is greater than or equal to another, up to a constant factor.
- Likewise, we say that $f(n)$ is $\Theta(g(n))$ (pronounced “ $f(n)$ is big-Theta of $g(n)$ ”) if $f(n)$ is $O(g(n))$ and $f(n)$ is $\Omega(g(n))$; that is, there are real constants $c' > 0$ and $c'' > 0$, and an integer constant $n_0 \geq 1$ such that $c'g(n) \leq f(n) \leq c''g(n)$, for $n \geq n_0$.

Examples

Example 1: **$3 \log n + \log \log n$ is $\Omega(\log n)$**

•*Proof:* $3 \log n + \log \log n \geq 3 \log n$, for $n \geq 2$.

Example 2: **$3 \log n + \log \log n$ is $\Theta(\log n)$**

•*Proof:* $3 \log n + \log \log n \leq 4 \log n$, for $n \geq 2$. Note that $\log \log n$ is not even defined for $n = 1$, but $\log \log n < \log n$, for $n \geq 2$. That is why we use $n \geq 2$.

•*Proof:* $3 \log n + \log \log n \geq 3 \log n$, for $n \geq 2$.

Little-Oh and Little-Omega

- Let $f(n)$ and $g(n)$ be functions mapping integers to real numbers. We say that $f(n)$ is $o(g(n))$ (pronounced “ $f(n)$ is little-oh of $g(n)$ ”) if, for any constant $c > 0$, there is a constant $n_0 \geq 1$ such that $f(n) < cg(n)$ for $n \geq n_0$.
- Likewise, we say that $f(n)$ is $\omega(g(n))$ (pronounced “ $f(n)$ is little-omega of $g(n)$ ”) if $g(n)$ is $o(f(n))$, that is, if, for any constant $c > 0$, there is a constant $n_0 \geq 1$ such that $f(n) > cg(n)$ for $n \geq n_0$.

Examples

- $x^2 \in O(x^2)$
- $x^2 \notin o(x^2)$
- $x^2 \in o(x^3)$

The following are true for Big-O, but would not be true if you used little-o:

- $x^2 \in O(x^2)$
- $x^2 \in O(x^2 + x)$
- $x^2 \in O(200 * x^2)$

The following are true for little-o:

- $x^2 \in o(x^3)$
- $x^2 \in o(x!)$
- $\ln(x) \in o(x)$

For example, the function $f(n) = 3n$ is:

- in $O(n^2)$, $o(n^2)$, and $O(n)$
- not in $O(\lg n)$, $o(\lg n)$, or $o(n)$

Limit Theorem

Notation	Common name	Limit test (note limit may not exist)
$f(n) \in O(g(n))$	Asymptotic upper bound	$\lim_{x \rightarrow \infty} \left \frac{f(x)}{g(x)} \right < \infty$
$f(n) \in o(g(n))$	Asymptotically negligible	$\lim_{x \rightarrow \infty} \left \frac{f(x)}{g(x)} \right = 0$
$f(n) \in \Omega(g(n))$	Asymptotic lower bound	$\lim_{x \rightarrow \infty} \left \frac{f(x)}{g(x)} \right > 0$
$f(n) \in \omega(g(n))$	Asymptotically dominant	$\lim_{x \rightarrow \infty} \left \frac{f(x)}{g(x)} \right = \infty$
$f(n) \in \Theta(g(n))$	Asymptotically tight bound	$0 < \lim_{x \rightarrow \infty} \left \frac{f(x)}{g(x)} \right < \infty$

Prove using limit theorem

- Example 1: $3n^2+2n$ is $O(n^2)$

Prove using limit theorem

- Example 2: $3n^2+2n$ is $o(n^2)$
- Example 3: $3n^2+2n$ is $o(n^3)$

Prove using limit theorem

- Example 4: $3n^2+2n$ is $\Omega(n)$
- Example 5: $3n^2+2n$ is $\omega(n)$

Asymptotically tight notation

- Theta, commonly written as Θ , is an Asymptotic Notation to denote the *asymptotically tight bound* on the growth rate of runtime of an algorithm.
- For example, if $f(n)$ is $O(g(n))$, and $f(n)$ is $O(h(n))$ and if $g(n)$ is $O(h(n))$ then $g(n)$ is called the asymptotically tight notation.

Asymptotically tight notation

- So for $\log(n)$ we could say:
 $\log(n)$ is $O(n^n)$ since $\log(n)$ grows asymptotically no faster than n^n
 $\log(n)$ is $O(n)$ since $\log(n)$ grows asymptotically no faster than n
 $\log(n)$ is $O(\log(n))$ since $\log(n)$ grows asymptotically no faster than $\log(n)$
We went from **loose upper bounds to a tight upper bound**
- $\log(n)$ is $\Omega(1)$ since $\log(n)$ grows asymptotically no slower than 1
 $\log(n)$ is $\Omega(\log(\log(n)))$ since $\log(n)$ grows asymptotically no slower than $\log(\log(n))$
 $\log(n)$ is $\Omega(\log(n))$ since $\log(n)$ grows asymptotically no slower than $\log(n)$
We went from **loose lower bounds to a tight lower bound**
- Since we have $\log(n)$ is $O(\log(n))$ and $\Omega(\log(n))$ we can say that $\log(n)$ is $\Theta(\log(n))$.

For the functions, n^k and c^n , what is the asymptotic relationship between these functions?

Assume that $k \geq 1$ and $c > 1$ are constants.

Choose all answers that apply:

☒ (A) n^k is $O(c^n)$ ✓

☐ (B) n^k is $\Omega(c^n)$

☐ (C) n^k is $\Theta(c^n)$

For the functions, $\log_2 n$ and $\log_8 n$, what is the asymptotic relationship between these functions?

Choose all answers that apply:

☒ (A) $\log_2 n$ is $O(\log_8 n)$ ✓

☒ (B) $\log_2 n$ is $\Omega(\log_8 n)$ ✓

☒ (C) $\log_2 n$ is $\Theta(\log_8 n)$ ✓

For the functions, 8^n and 4^n , what is the asymptotic relationship between these functions?

Choose all answers that apply:

☐ A 8^n is $O(4^n)$

☒ B 8^n is $\Omega(4^n)$ ✓

☐ C 8^n is $\Theta(4^n)$

Ordering functions by growth rate

Some Functions Ordered by Growth Rate	Common Name
$\log n$	logarithmic
$\log^2 n$	polylogarithmic
$\sqrt{n}$	square root
n	linear
$n \log n$	linearithmic
n^2	quadratic
n^3	cubic
2^n	exponential

Ordering functions by growth rate

n	$\log n$	$\log^2 n$	$\sqrt{n}$	$n \log n$	n^2	n^3	2^n
4	2	4	2	8	16	64	16
16	4	16	4	64	256	4,096	65,536
64	6	36	8	384	4,096	262,144	1.84×10^{19}
256	8	64	16	2,048	65,536	16,777,216	1.15×10^{77}
1,024	10	100	32	10,240	1,048,576	1.07×10^9	1.79×10^{308}
4,096	12	144	64	49,152	16,777,216	6.87×10^{10}	10^{1233}
16,384	14	196	128	229,376	268,435,456	4.4×10^{12}	10^{4932}
65,536	16	256	256	1,048,576	4.29×10^9	2.81×10^{14}	10^{19728}
262,144	18	324	512	4,718,592	6.87×10^{10}	1.8×10^{16}	10^{78913}

- Note the point at which the function $\text{sqrt}(n)$ dominates $\log^2 n$.

Logarithmic and Exponent properties

- Let a , b , and c be positive real numbers. We have

1. $\log_b ac = \log_b a + \log_b c$

2. $\log_b a/c = \log_b a - \log_b c$

3. $\log_b a^c = c \log_b a$

4. $\log_b a = (\log_c a) / \log_c b$

5. $b^{\log_c a} = a^{\log_c b}$

6. $(b^a)^c = b^{ac}$

7. $b^a b^c = b^{a+c}$

8. $b^a / b^c = b^{a-c}.$

Try yourself!!

- Order the following growth rates in ascending order

$$\begin{array}{ccccc} 6n \log n & 2^{100} & \log \log n & \log^2 n & 2^{\log n} \\ 2^{2^n} & \lceil \sqrt{n} \rceil & n^{0.01} & 1/n & 4n^{3/2} \end{array}$$

- Order these asymptotic notations by increasing growth rate.

$$n^{1/\log n}, \quad e^n, \quad \left(\frac{3}{2}\right)^n, \quad n^{\log \log n}, \quad (\log n)^{\log n}$$

Solving Recurrences

- An equation or inequality that describes a function in terms of its value on smaller inputs.

$$T(n) = T(n-1) + n$$

- Recurrences arise when an algorithm contains recursive calls to itself
- What is the actual running time of the algorithm?
- Need to solve the recurrence
 - Find an explicit formula of the expression
 - Bound the recurrence by an expression that involves n

Example Recurrences

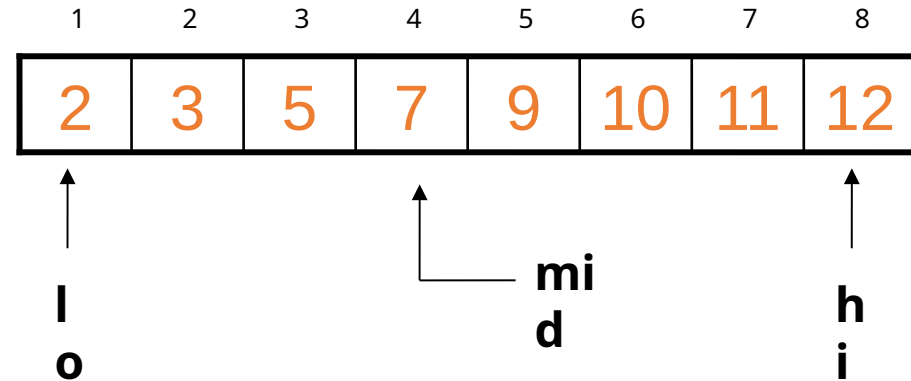
- $T(n) = T(n-1) + n$ $\Theta(n^2)$
 - Recursive algorithm that loops through the input to eliminate one item
- $T(n) = T(n/2) + c$ $\Theta(\lg n)$
 - Recursive algorithm that halves the input in one step
- $T(n) = T(n/2) + n$ $\Theta(n)$
 - Recursive algorithm that halves the input but must examine every item in the input
- $T(n) = 2T(n/2) + 1$ $\Theta(n)$
 - Recursive algorithm that splits the input into 2 halves and does a constant amount of other work

Recurrent Algorithms - BINARY-SEARCH

- for an ordered array A, finds if x is in the array A[lo...hi]

Alg.: BINARY-SEARCH (A, lo, hi, x)

```
if (lo > hi)
    return FALSE
mid  $\leftarrow \lfloor (lo+hi)/2 \rfloor$ 
if x = A[mid]
    return TRUE
if ( x < A[mid] )
    BINARY-SEARCH (A, lo, mid-1, x)
if ( x > A[mid] )
    BINARY-SEARCH (A, mid+1, hi, x)
```



Example

- $A[8] = \{1, 2, 3, 4, 5, 7, 9, 11\}$
 - $lo = 1$ $hi = 8$ $x = 7$

1	2	3	4	5	6	7	8
1	2	3	4	5	7	9	11

$mid = 4, lo = 5, hi = 8$

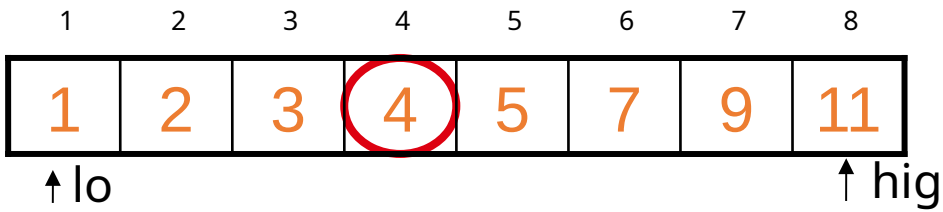
				5	6	7	8
1	2	3	4	5	7	9	11

$mid = 6, A[mid] = x$
Found!

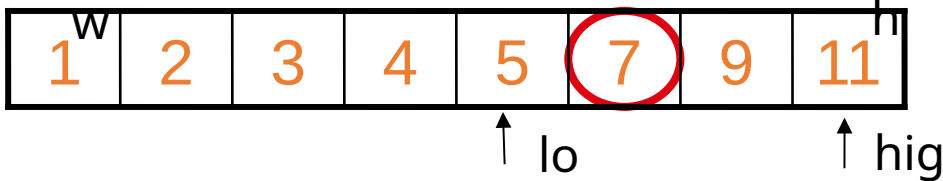
Another Example

- $A[8] = \{1, 2, 3, 4, 5, 7, 9, 11\}$

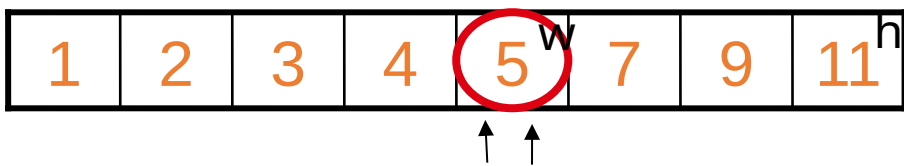
– lo = 1 hi = 8 **x = 6**



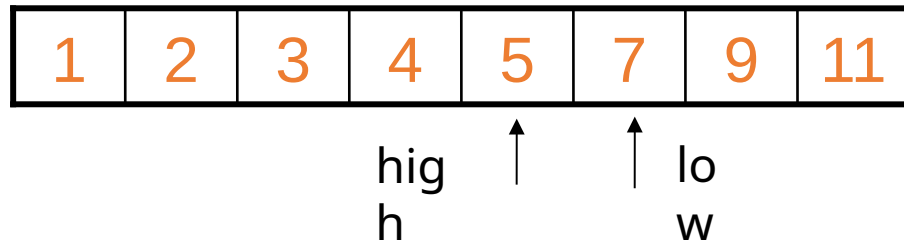
mid = 4, lo = 5, hi = 8



mid = 6, A[6] = 7, lo = 5, hi = 5



mid = 5, A[5] = 5, lo = 6, hi = 5
NOT FOUND!



Analysis of BINARY-SEARCH

Alg.: BINARY-SEARCH (A, lo, hi, x)

if ($lo > hi$)

return FALSE

$mid \leftarrow \lfloor (lo+hi)/2 \rfloor$

if $x = A[mid]$

return TRUE

if ($x < A[mid]$)

BINARY-SEARCH (A, lo, mid-1, x) same problem of size $n/2$

if ($x > A[mid]$)

BINARY-SEARCH (A, mid+1, hi, x) same problem of size $n/2$

- $T(n) = c + T(n/2)$

- $T(n)$ – running time for an array of size n

Methods for Solving Recurrences

- Iteration method (or) Iterative substitution method
- Substitution method (or) Guess and test method
- Recursion tree method
- Master method

The Iteration Method

- Convert the recurrence into a summation and try to bound it using known series
 - Iterate the recurrence until the initial condition is reached.
 - Use back-substitution to express the recurrence in terms of n and the initial (boundary) condition.

The Iteration Method – Example 1

Example: $T(n) = c + T(n/2)$

$$T(n) = c + T(n/2)$$

$$= c + c + T(n/4)$$

$$= c + c + c + T(n/8)$$

Assume $n = 2^k$

$$T(n) = c + \underbrace{c + \dots + c}_{k \text{ times}} + T(1)$$

$$= c \lg n + T(1)$$

$$= \Theta(\lg n)$$

$$T(n/2) = c + T(n/4)$$

$$T(n/4) = c + T(n/8)$$

Iteration Method – Example 2

$$\text{Example: } T(n) = n + 2T(n/2)$$

Assume: $n = 2^k$

$$T(n) = n + 2T(n/2)$$

$$T(n/2) = n/2 + 2T(n/4)$$

$$= n + 2(n/2 + 2T(n/4))$$

$$= n + n + 4T(n/4)$$

$$= n + n + 4(n/4 + 2T(n/8))$$

$$= n + n + n + 8T(n/8)$$

$$\dots = in + 2^i T(n/2^i)$$

$$= kn + 2^k T(1)$$

$$= n \lg n + nT(1) = \Theta(n \lg n)$$

Iteration Method - Example 3

$$T(n) = 8T(n/2) + n^2$$

$$\begin{aligned}
 T(n) &= 8T(n/2) + n^2 \quad (1^{\text{st}} \text{ level}) \\
 &= 8 \left[8T(n/2^2) + \left(\frac{n}{2}\right)^2 \right] + n^2 \\
 &= 8^2 T(n/2^2) + 3n^2 \quad (2^{\text{nd}} \text{ level}) \\
 &= 8^2 \left[8T(n/2^3) + \left(\frac{n}{2^2}\right)^2 \right] + 3n^2 \\
 &= 8^3 T(n/2^3) + 7n^2 \quad (3^{\text{rd}} \text{ level}) \\
 &= 8^{i^0} T\left(\frac{n}{2^{i^0}}\right) + (2^{i^0} - 1)n^2 \quad (i^{\text{th}} \text{ level}).
 \end{aligned}$$

$$\begin{aligned}
 &= 8^{i^0} T\left(\frac{n}{2^{i^0}}\right) + (2^{i^0} - 1)n^2 \quad (i^{\text{th}} \text{ level}). \\
 &= 8^{\log_2 n} T(1) + (n-1)n^2 \\
 &= n^{\log_2 8} a + n^3 - n^2 \\
 &= an^3 + n^3 - n^2 = n^3(a+1) - n^2 \\
 &\sim O(n^3).
 \end{aligned}$$

$$\begin{aligned}
 \frac{n}{2^i} = 1 &\Rightarrow n = 2^i \\
 i &= \log_2 n
 \end{aligned}$$

Iteration Method – Example 4

$$T(n) = 2T(n/2) + b$$

Iteration Method – Example 5

$$T(n) = 2T(n-1) + b \quad (\text{Tower of Hanoi})$$

$$\begin{aligned} T(n) &= 2T(n-1) + b. \quad (1^{\text{st}} \text{ level}) \\ &= 2[2T(n-2) + b] + b \\ &= 2^2 T(n-2) + 3b. \quad (2^{\text{nd}} \text{ level}) \\ &= 2^2 [2T(n-3) + b] + 3b \\ &= 2^3 T(n-3) + 7b. \quad (3^{\text{rd}} \text{ level}) \\ &= 2^i T(n-i) + (2^i - 1)b. \quad (i^{\text{th}} \text{ level}) \end{aligned}$$

$$\begin{aligned} &= 2^i T(n-i) + (2^i - 1)b. \quad (i^{\text{th}} \text{ level}) \\ &= 2^{n-1} a + (2^{n-1} - 1)b. \quad \boxed{\begin{matrix} n-i=1 \\ n-1=i \end{matrix}} \\ &= 2^{n-1} (a+b) - b. \\ &\approx O(2^{n-1}) \quad \text{or} \quad \approx O(2^n). \end{aligned}$$

Iteration Method - Example 6

$$T(n) = T(n-1) + c \quad \text{1st level}$$

$$= [T(n-2) + c] + c$$

$$= T(n-2) + 2c$$

2nd level

$$= [T(n-3) + c] + 2c$$

$$= T(n-3) + 3c$$

3rd level

$$\vdots$$
$$= T(n-i) + ic$$

ith level

$$= T(1) + (n-1)c$$

$$n-i = 1$$

$$n-1 = i$$

$$= a + cn - c$$

$$\approx O(n)$$

Iteration Method - Example 7

$$\begin{aligned}T(n) &= T(n-1) + n && \text{1st level} \\&= T(n-2) + n-1 + n && \text{2nd level} \\&= T(n-2) + 2n-1 \\&= T(n-3) + (n-2) + 2n-1 && \text{3rd level} \\&= T(n-3) + 3n-3 \\&= T(n-4) + (n-3) + 3n-3 && \text{4th level} \\&= T(n-4) + 4n-6 \\i^{\text{th}} \text{ level} &= T(n-i) + i^{\circ}n - \frac{i(i-1)}{2} \\&= T(1) + n(n-1) + \frac{(n-1)(n-2)}{2} \\&\approx O(n^2)\end{aligned}$$

[1, 3, 6, ...] sum of series

$$\begin{aligned}n-i^{\circ} &= 1 \\n-1 &= i^{\circ}\end{aligned}$$

Iteration Method - Example 8

$$\begin{aligned}
 T(n) &= T(n/2) + n && [1^{\text{st}} \text{ level}] \\
 &= T(n/2^2) + \frac{n}{2} + n \\
 &= T(n/2^3) + \frac{n}{2} + \frac{n}{2} + n && 2^{\text{nd}} \text{ level} \\
 &= T(n/2^3) + \frac{n}{2} + \frac{n}{2} + n. \\
 &= T(n/2^3) + n \left[1 + \frac{1}{2} + \frac{1}{2^2} \right] && 3^{\text{rd}} \text{ level}
 \end{aligned}$$

at i^{th} level

$$= T(n/2^i) + n \left[1 + \frac{1}{2} + \frac{1}{2^2} + \dots + \frac{1}{2^{i-1}} \right]$$

$$= T(1) + n \left[1 + \frac{1}{2} + \frac{1}{2^2} + \dots + \frac{1}{2^{i-1}} \right]$$

But this series is less than infinite series
Infinite Geometric decreasing series is

$$\frac{a}{1-r} = \frac{1}{1-1/2} = 2$$

$$\Rightarrow T(1) + n(2)$$

$$\leq a + 2n$$

$$(i.e.) \underline{\underline{O(n)}}$$

↓
Geometric series.

$$\begin{aligned}
 \frac{n}{2^i} &= 1 \\
 n &= 2^i \\
 i &= \log_2 n
 \end{aligned}$$

$$\begin{aligned}
 \frac{a(1-r^n)}{1-r} &= \frac{1(1-(1/2)^{2/n})}{1-1/2} \\
 &= 2 \left(1 - \frac{1}{2^{2/n}} \right)
 \end{aligned}$$

Iteration Method – Example 9

$$T(n) = 2T(n/2) + 1 \quad 1^{\text{st}} \text{ level.}$$

$$= 2[2T(n/2) + 1] + 1$$

$$= 2^2 T(n/2^2) + 3 \quad 2^{\text{nd}} \text{ level.}$$

$$= 2^2 [2T(n/2^3) + 1] + 3$$

$$= 2^3 T(n/2^3) + 7 \quad 3^{\text{rd}} \text{ level.}$$

$$\therefore = 2^i T(n/2^i) + 2^i - 1 \quad i^{\text{th}} \text{ level.}$$

$$= na + n - 1$$

$$= n(a+1) - 1$$

$$\simeq O(n).$$

$$\boxed{\begin{array}{l} n = 2^i \\ i = \log_2 n \end{array}}$$

The substitution method

1. Guess a solution
2. Use induction to prove that the solution works

Substitution method

- Guess a solution
 - $T(n) = O(g(n))$
 - Induction goal: **apply the definition of the asymptotic notation**
 - $T(n) \leq d g(n)$, for some $d > 0$ and $n \geq n_0$ (strong induction)
 - Induction hypothesis: $T(k) \leq d g(k)$ for all $k < n$
- Prove the induction goal
 - Use the **induction hypothesis** to **find some values of the constants d and n_0** for which the **induction goal** holds

Example: Binary Search

$$T(n) = c + T(n/2)$$

- Guess: $T(n) = O(\lg n)$
 - Induction goal: $T(n) \leq d \lg n$, for some d and $n \geq n_0$
 - Induction hypothesis: $T(n/2) \leq d \lg(n/2)$
- Proof of induction goal:

$$T(n) = T(n/2) + c \leq d \lg(n/2) + c$$

$$= d \lg n - d + c \leq d \lg n$$

$$\text{if: } -d + c \leq 0, d \geq c$$

- Base case?

Example 2

$$T(n) = T(n-1) + n$$

- Guess: $T(n) = O(n^2)$
 - Induction goal: $T(n) \leq c n^2$, for some c and $n \geq n_0$
 - Induction hypothesis: $T(n-1) \leq c(n-1)^2$ for all $k < n$
- Proof of induction goal:

$$T(n) = T(n-1) + n \leq c(n-1)^2 + n$$

$$= cn^2 - (2cn - c - n) \leq cn^2$$

$$\text{if: } 2cn - c - n \geq 0 \Leftrightarrow c \geq n/(2n-1) \Leftrightarrow c \geq 1/(2 - 1/n)$$

- For $n \geq 1 \Rightarrow 2 - 1/n \geq 1 \Rightarrow$ any $c \geq 1$ will work

Example 3

$$T(n) = 2T(n/2) + n$$

- Guess: $T(n) = O(n \lg n)$
 - Induction goal: $T(n) \leq cn \lg n$, for some c and $n \geq n_0$
 - Induction hypothesis: $T(n/2) \leq cn/2 \lg(n/2)$
- Proof of induction goal:

$$T(n) = 2T(n/2) + n \leq 2c (n/2) \lg(n/2) + n$$

$$= cn \lg n - cn + n \leq cn \lg n$$

$$\text{if: } -cn + n \leq 0 \Rightarrow c \geq 1$$

- Base case?

Example 4

- $T(n) = 2T(n/2) + bn \log n$

First Guess: $T(n) \leq cn \log n$

$$\begin{aligned}T(n) &= 2T(n/2) + bn \log n \\&\leq 2(c(n/2) \log(n/2)) + bn \log n \\&= cn(\log n - \log 2) + bn \log n \\&= cn \log n - cn + bn \log n.\end{aligned}$$

But there is no way that we can make this last line less than or equal to $cn \log n$ for $n \geq 2$. Thus, this first guess was not sufficient

Better Guess: $T(n) \leq cn \log^2 n$

$$\begin{aligned}T(n) &= 2T(n/2) + bn \log n \\&\leq 2(c(n/2) \log^2(n/2)) + bn \log n \\&= cn(\log^2 n - 2 \log n + 1) + bn \log n \\&= cn \log^2 n - 2cn \log n + cn + bn \log n \\&\leq cn \log^2 n,\end{aligned}$$

provided $c \geq b$

The recursion-tree method

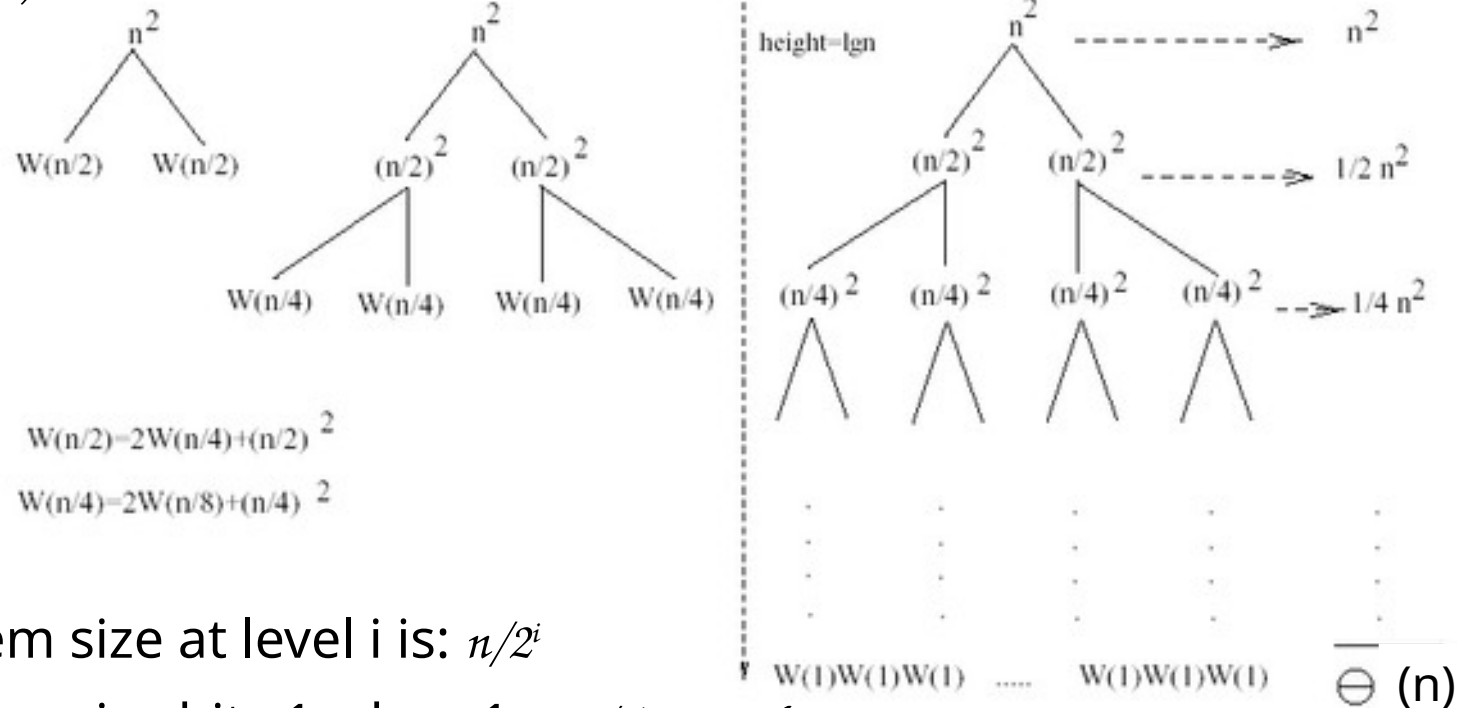
Convert the recurrence into a tree:

- Each node represents the cost incurred at various levels of recursion
- Sum up the costs of all levels

Used to “guess” a solution for the recurrence

Example 1

$$\mathcal{W}(n) = 2\mathcal{W}(n/2) + n^2$$

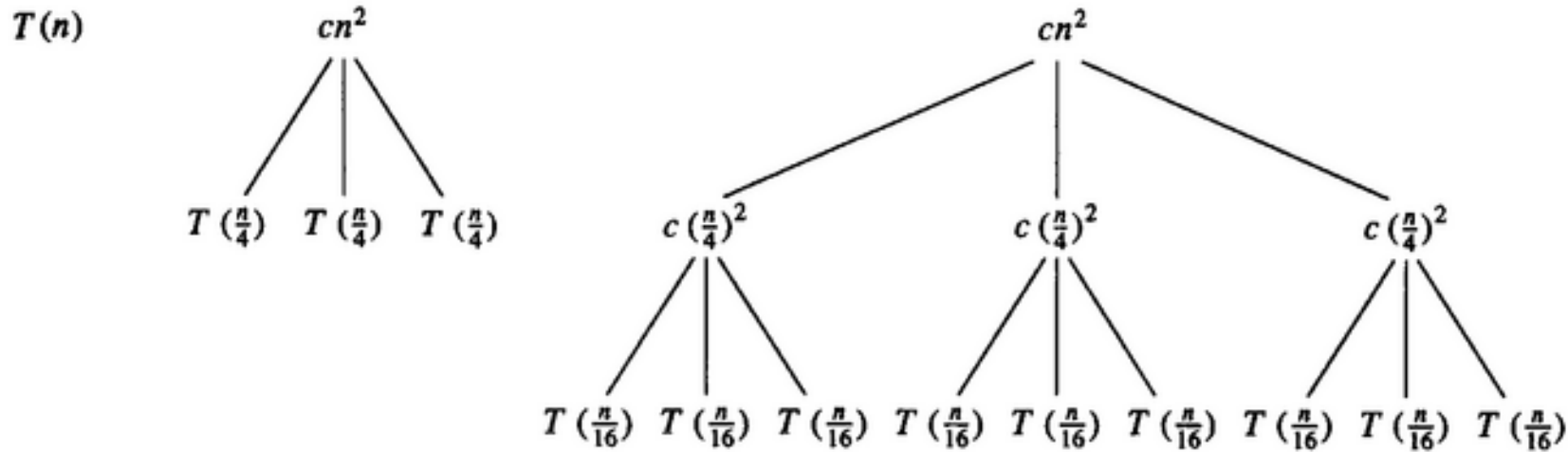


- Subproblem size at level i is: $n/2^i$
- Subproblem size hits 1 when $1 = n/2^i \Rightarrow i = \lg n$
- Cost of the problem at level $i = (n/2^i)^2$ No. of nodes at level $i = 2^i$
- Total cost:
$$\mathcal{W}(n) = \sum_{i=0}^{\lg n - 1} \frac{n^2}{2^i} + 2^{\lg n} \mathcal{W}(1) = n^2 \sum_{i=0}^{\lg n - 1} \left(\frac{1}{2}\right)^i + n \leq n^2 \sum_{i=0}^{\infty} \left(\frac{1}{2}\right)^i + O(n) = n^2 \frac{1}{1 - 1/2} + O(n) = 2n^2$$

$$\Rightarrow \mathcal{W}(n) = O(n^2)$$

Example 2

E.g.: $T(n) = 3T(n/4) + cn^2$



- Subproblem size at level i is: $n/4^i$
- Subproblem size hits 1 when $1 = n/4^i \Rightarrow i = \log_4 n$
- Cost of a node at level $i = c(n/4^i)^2$
- Number of nodes at level $i = 3^i \Rightarrow$ last level has $3^{\log_4 n} = n^{\log_4 3}$ nodes
- Total cost:

$$T(n) = \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16} \right)^i cn^2 + \Theta(n^{\log_4 3}) \leq \sum_{i=0}^{\infty} \left(\frac{3}{16} \right)^i cn^2 + \Theta(n^{\log_4 3}) = \frac{1}{1 - \frac{3}{16}} cn^2 + \Theta(n^{\log_4 3}) = O(n^2)$$

$$\Rightarrow T(n) = O(n^2)$$

Example 2 - Substitution

$$T(n) = 3T(n/4) + cn^2$$

- Guess: $T(n) = O(n^2)$
 - Induction goal: $T(n) \leq dn^2$, for some d and $n \geq n_0$
 - Induction hypothesis: $T(n/4) \leq d(n/4)^2$
- Proof of induction goal:

$$T(n) = 3T(n/4) + cn^2$$

$$\leq 3d(n/4)^2 + cn^2$$

$$= (3/16)d n^2 + cn^2$$

$$\leq d n^2 \quad \text{if: } d \geq (16/13)c$$

- Therefore: $T(n) = O(n^2)$

Example 3 (simpler proof)

$$\mathcal{W}(n) = \mathcal{W}(n/3) + \mathcal{W}(2n/3) + n$$

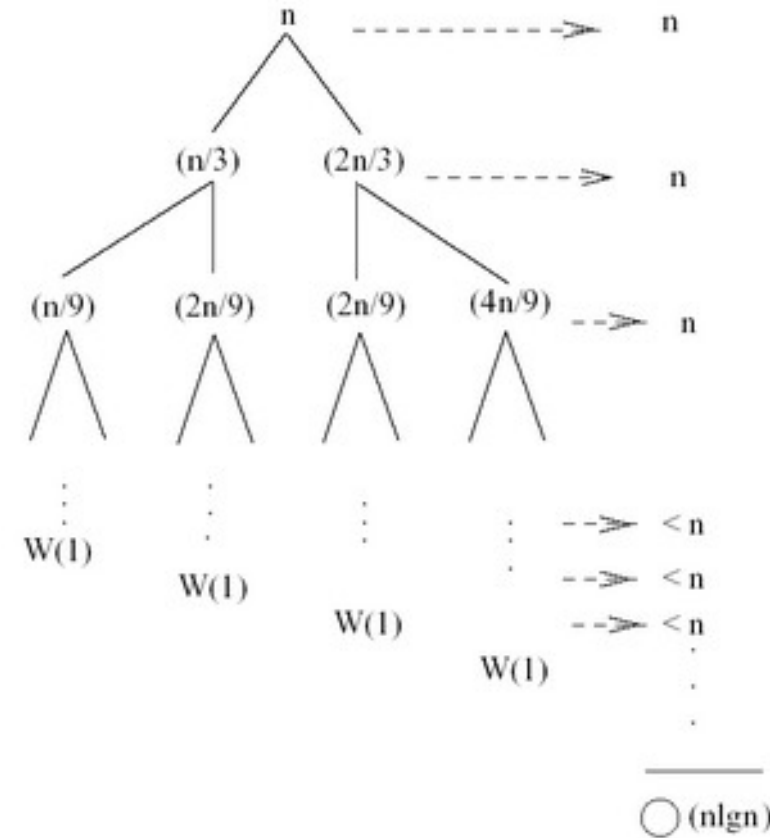
- The longest path from the root to a leaf is:

$$n \rightarrow (2/3)n \rightarrow (2/3)^2 n \rightarrow \dots \rightarrow 1$$

- Subproblem size hits 1 when
- $1 = (2/3)^i n \Leftrightarrow i = \log_{3/2} n$
- Cost of the problem at level $i = n$
- Total cost:

$$W(n) < n + n + \dots = n(\log_{3/2} n) = n \frac{\lg n}{\lg \frac{3}{2}} = O(n \lg n)$$

$$\Rightarrow \mathcal{W}(n) = O(n \lg n)$$



Example 3

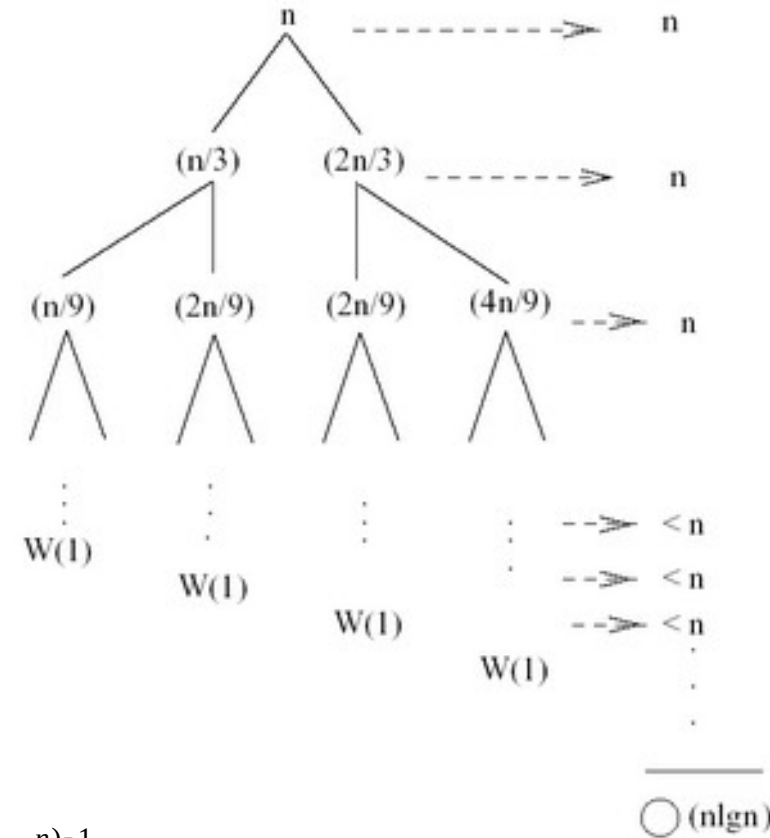
$$\mathcal{W}(n) = \mathcal{W}(n/3) + \mathcal{W}(2n/3) + n$$

- The longest path from the root to a leaf is:

$$n \rightarrow (2/3)n \rightarrow (2/3)^2 n \rightarrow \dots \rightarrow 1$$

- Subproblem size hits 1 when
- $1 = (2/3)^i n \Leftrightarrow i = \log_{3/2} n$
- Cost of the problem at level $i = n$
- Total cost:

$$\begin{aligned} W(n) &< n + n + \dots = \sum_{i=0}^{(\log_{3/2} n) - 1} n + 2^{(\log_{3/2} n)} W(1) < \\ &< n \sum_{i=0}^{\log_{3/2} n} 1 + n^{\log_{3/2} 2} = n \log_{3/2} n + O(n) = n \frac{\lg n}{\lg 3/2} + O(n) = \frac{1}{\lg 3/2} n \lg n + O(n) \\ &\Rightarrow \mathcal{W}(n) = O(n \lg n) \end{aligned}$$



Example 3 - Substitution

$$\mathcal{W}(n) = \mathcal{W}(n/3) + \mathcal{W}(2n/3) + O(n)$$

- Guess: $\mathcal{W}(n) = O(n \lg n)$
 - Induction goal: $\mathcal{W}(n) \leq d n \lg n$, for some d and $n \geq n_0$
 - Induction hypothesis: $\mathcal{W}(k) \leq d k \lg k$ for any $k < n$
($n/3, 2n/3$)
- Proof of induction goal:

Try it out as an exercise!!
- $T(n) = O(n \lg n)$

Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:

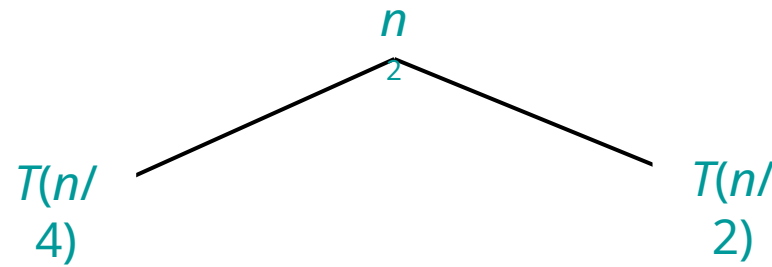
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:

$T(n)$
)

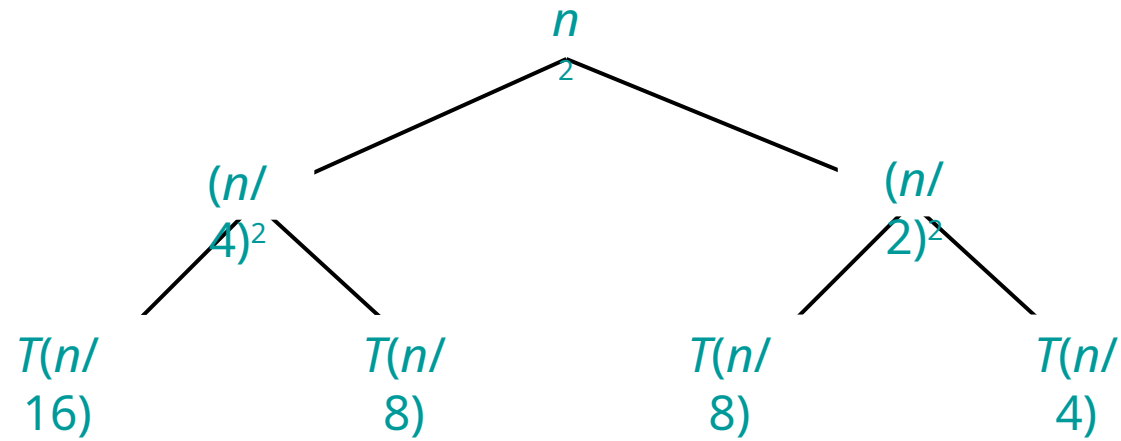
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



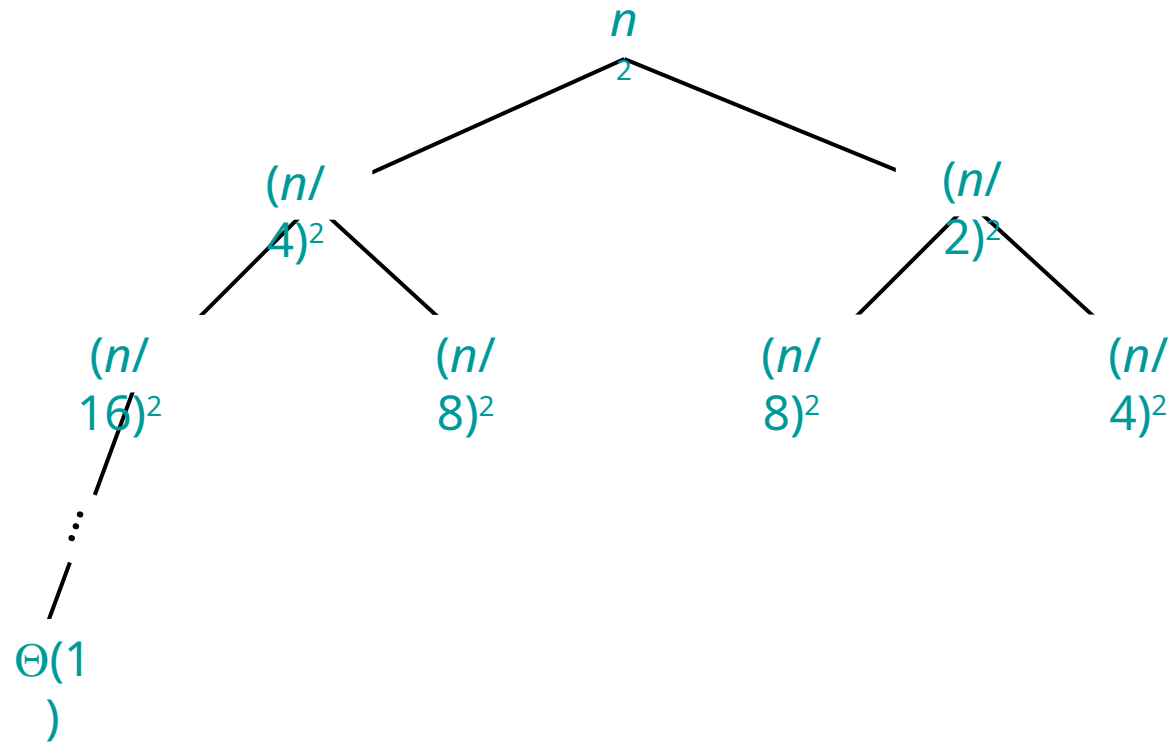
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



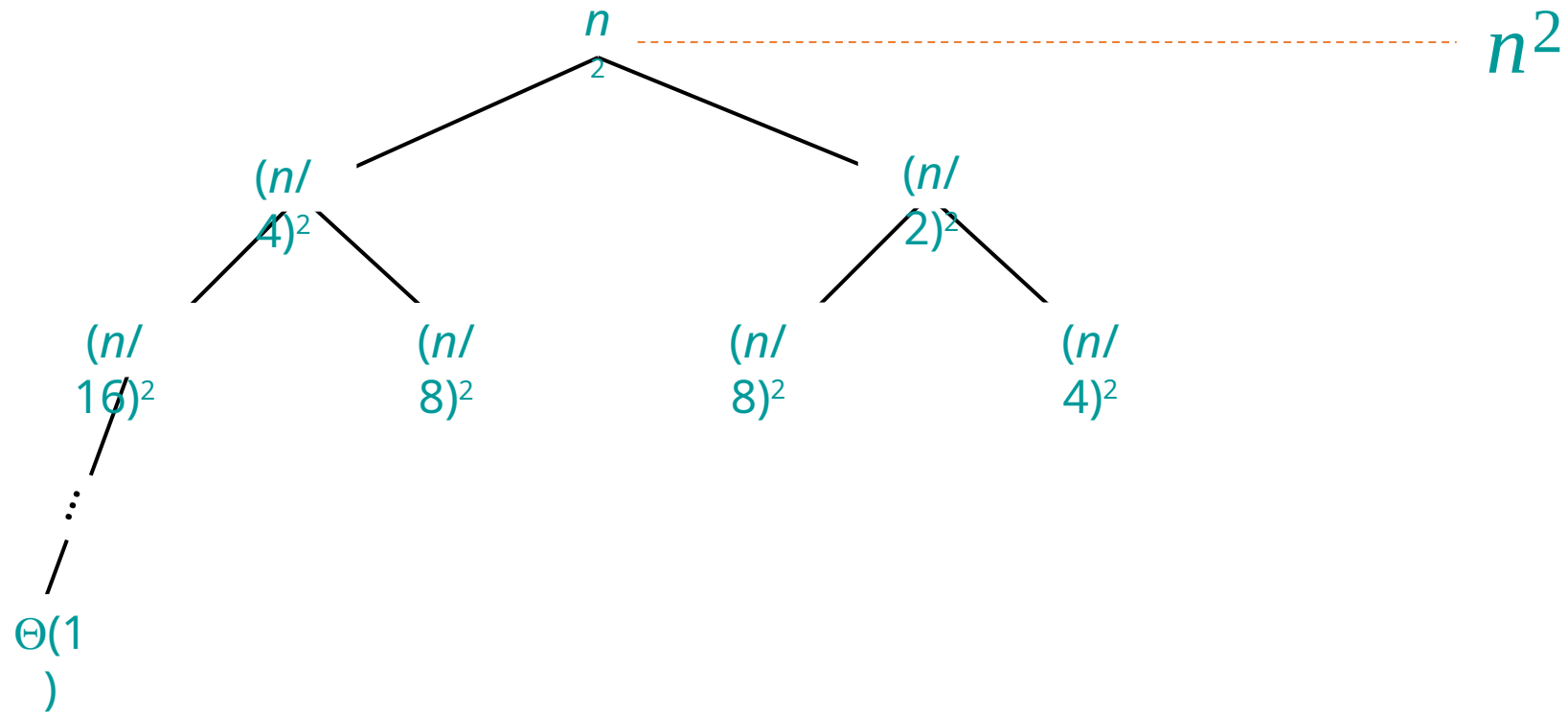
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



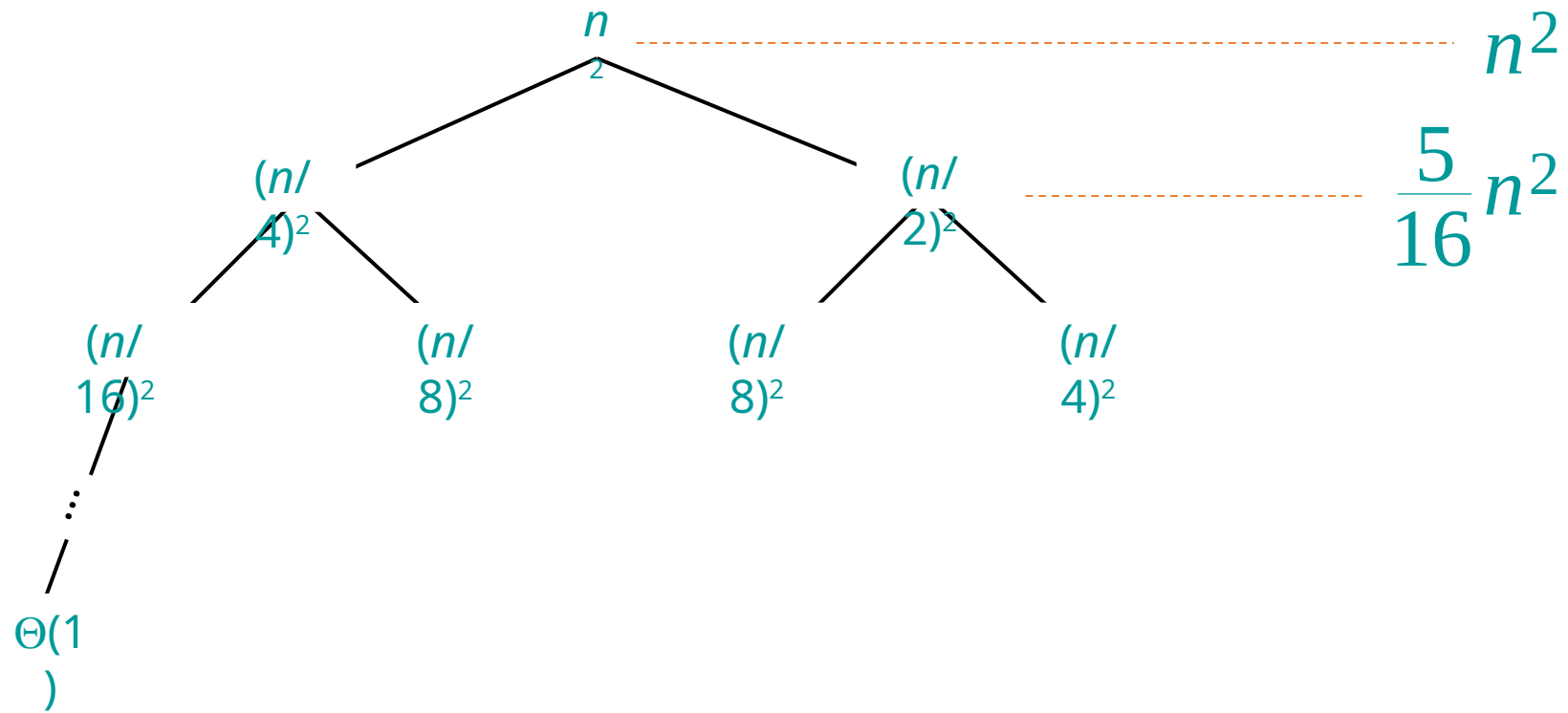
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



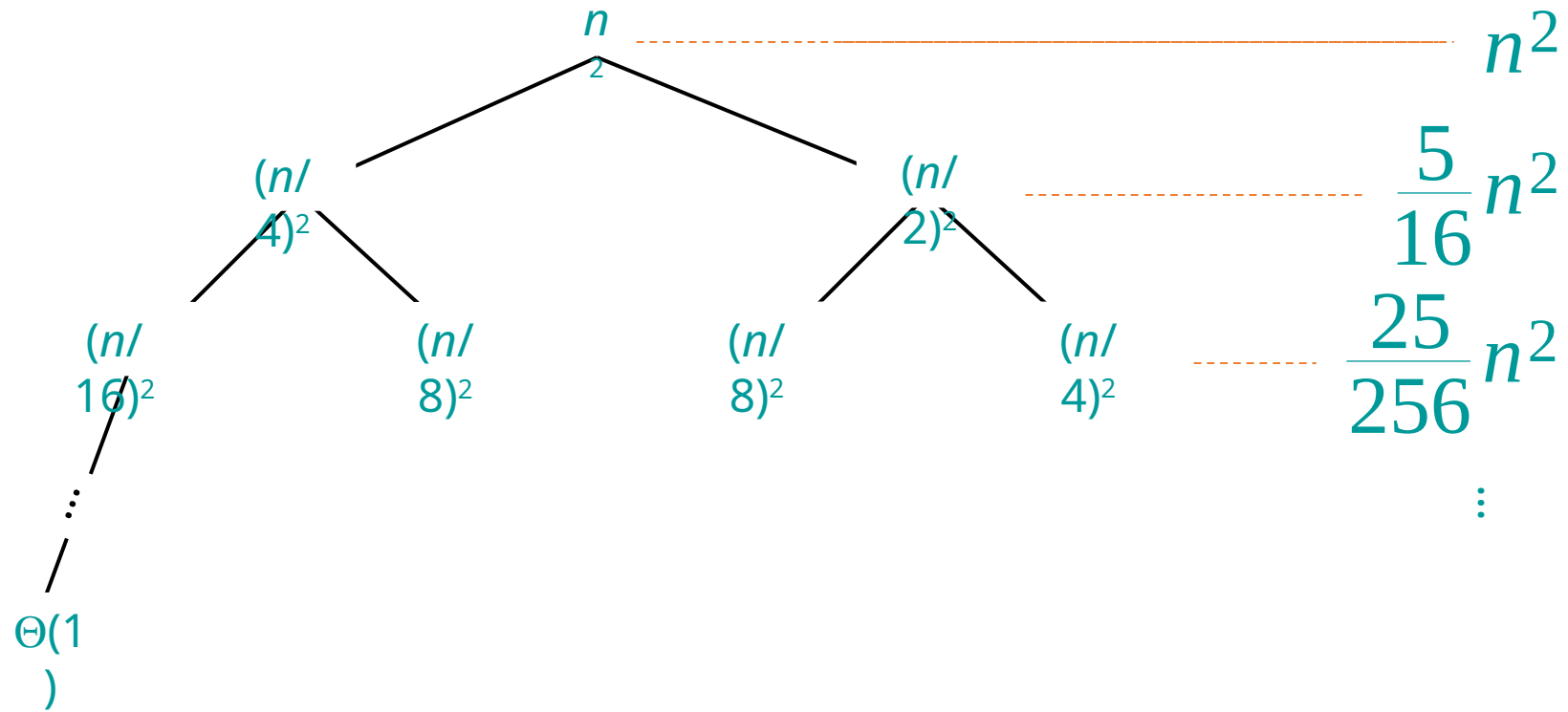
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



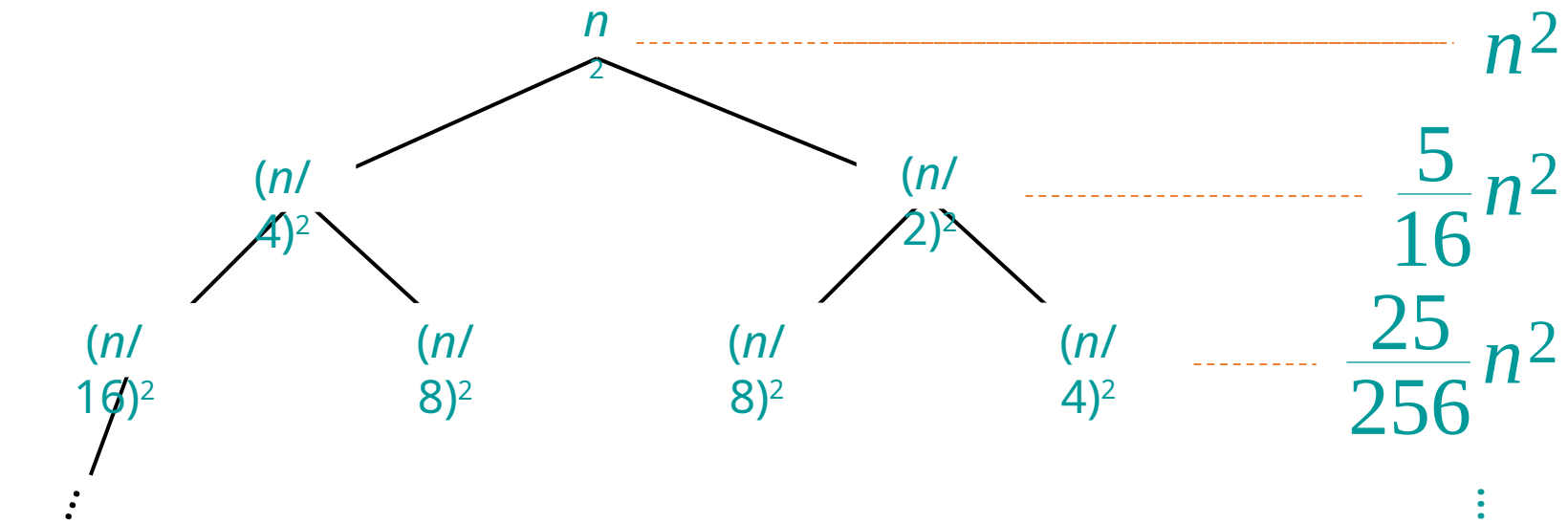
Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



Example of recursion tree

Solve $T(n) = T(n/4) + T(n/2) + n^2$:



$$\begin{aligned} \text{Total} &= n^2 \left(1 + \frac{5}{16} + \left(\frac{5}{16}\right)^2 + \left(\frac{5}{16}\right)^3 + \dots \right) \\ &= \Theta(n^2) \end{aligned}$$

geometric series



Appendix: geometric series

$$1 + x + x^2 + \dots + x^n = \frac{1 - x^{n+1}}{1 - x} \quad \text{for } x \neq 1$$

$$1 + x + x^2 + \dots = \frac{1}{1 - x} \quad \text{for } |x| < 1$$

Master's method

- “Cookbook” for solving recurrences of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where, $a \geq 1$, $b > 1$, and $f(n) > 0$

Idea: compare $f(n)$ with $n^{\log_b a}$

- $f(n)$ is asymptotically smaller or larger than $n^{\log_b a}$ by a polynomial factor n^ϵ
- $f(n)$ is asymptotically equal with $n^{\log_b a}$

Master's method

- “Cookbook” for solving recurrences of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

where, $a \geq 1$, $b > 1$, and $f(n) > 0$

Case 1: if $f(n) = O(n^{\log_b a - \varepsilon})$ for some $\varepsilon > 0$, then: $T(n) = \Theta(n^{\log_b a})$

Case 2: if $f(n) = \Theta(n^{\log_b a})$, then: $T(n) = \Theta(n^{\log_b a} \lg n)$

Case 3: if $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some $\varepsilon > 0$, and if

$af(n/b) \leq cf(n)$ for some $c < 1$ and all sufficiently large n , then:

regularity
condition

$$T(n) = \Theta(f(n))$$

Examples

$$T(n) = 2T(n/2) + n$$

$$a = 2, b = 2, \log_2 2 = 1$$

Compare $n^{\log_2 2}$ with $f(n) = n$

$\Rightarrow f(n) = \Theta(n) \Rightarrow \text{Case 2}$

$\Rightarrow T(n) = \Theta(n \lg n)$

Examples

$$T(n) = 2T(n/2) + n^2$$

$$a = 2, b = 2, \log_2 2 = 1$$

Compare n with $f(n) = n^2$

$\Rightarrow f(n) = \Omega(n^{1+\epsilon})$ Case 3 $\Rightarrow$ verify regularity cond.

$$a f(n/b) \leq c f(n)$$

$$\Leftrightarrow 2 n^2/4 \leq c n^2 \Rightarrow c = 1/2 \text{ is a solution } (c < 1)$$

$$\Rightarrow T(n) = \Theta(n^2)$$

Examples (cont.)

$$T(n) = 2T(n/2) + \sqrt{n}$$

$$a = 2, b = 2, \log_2 2 = 1$$

Compare n with $f(n) = n^{1/2}$

$$\Rightarrow f(n) = O(n^{1-\epsilon}) \quad \text{Case 1}$$

$$\Rightarrow T(n) = \Theta(n)$$

Examples

Ex. $T(n) = 4T(n/2) + n$

$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n.$$

CASE 1: $f(n) = O(n^{2-\epsilon})$ for $\epsilon = 1$.

$$\therefore T(n) = \Theta(n^2).$$

Ex. $T(n) = 4T(n/2) + n^2$

$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n^2.$$

CASE 2: $f(n) = \Theta(n^2 \lg^0 n)$, that is, $k = 0$.

$$\therefore T(n) = \Theta(n^2 \lg n).$$

Examples

Ex. $T(n) = 4T(n/2) + n^3$

$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n^3.$$

CASE 3: $f(n) = \Omega(n^{2+\epsilon})$ for $\epsilon = 1$

and $4(cn/2)^3 \leq cn^3$ (reg. cond.) for $c = 1/2$.

$$\therefore T(n) = \Theta(n^3).$$

Ex. $T(n) = 4T(n/2) + n^2/\lg n$

$$a = 4, b = 2 \Rightarrow n^{\log_b a} = n^2; f(n) = n^2/\lg n.$$

Master method does not apply. In particular,
for every constant $\epsilon > 0$, we have $n^\epsilon = \omega(\lg n)$.

Examples

$$T(n) = 3T(n/4) + n \lg n$$

$$a = 3, b = 4, \log_4 3 = 0.793$$

Compare $n^{0.793}$ with $f(n) = n \lg n$

$$f(n) = \Omega(n^{\log_4 3 + \epsilon}) \text{ Case 3}$$

Check regularity condition:

$$3 \star (n/4) \lg(n/4) \leq (3/4) n \lg n = c \star f(n), c=3/4$$

$$\Rightarrow T(n) = \Theta(n \lg n)$$

Examples

$$T(n) = 2T(n/2) + n \lg n$$

$$a = 2, b = 2, \log_2 2 = 1$$

- Compare n with $f(n) = n \lg n$
 - seems like case 3 should apply

Here $f(n) = n \log n$ and $n^{\log_b a} = n^{\log_2 2} = n^1 = n$. So $n^{\log_b a + \epsilon} = n^{1+\epsilon}$. But $n \log n$ is not $\Omega(n^{1+\epsilon})$.

So it is not under case 3. It is also not under second case: If $f(n) = \Theta(n^{\log_b a})$, then second case can be applied. But here $n^{\log_b a} = n$ and $f(n) = n \log n$ is not $\Theta(n)$.

This recurrence can't be solved by Master method.

Changing variables

$$T(n) = 2T(\sqrt{n}) + \lg n$$

- Rename: $m = \lg n \Rightarrow n = 2^m$

$$T(2^m) = 2T(2^{m/2}) + m$$

- Rename: $S(m) = T(2^m)$

$$S(m) = 2S(m/2) + m \Rightarrow S(m) = O(m \lg m) \text{ (demonstrated before)}$$

$$T(n) = T(2^m) = S(m) = O(m \lg m) = O(\lg n \lg \lg n)$$

Idea: transform the recurrence to one that you have seen before